

Plan détaillé — Mémoire Data Engineering

Sujet 2 · Les arbres de recherche et les index de base de données

Introduction

Problématique et cadrage

~1–2 pages

Accroche & contexte

- Une requête SQL sur 10 millions de lignes : 0.003 s avec index, 4.2 s sans — pourquoi ?
- L'index comme structure de données cachée, invisible mais déterminante en production
- CREATE INDEX : une ligne SQL qui cache une mécanique algorithmique sophistiquée

Problématique

- Quelles structures permettent d'indexer efficacement des millions d'enregistrements sur disque, et comment PostgreSQL les exploite pour optimiser les requêtes ?

Annonce du plan

- 3 parties : du BST naïf aux arbres équilibrés → du BST au B+tree → application PostgreSQL

Partie I

L'arbre binaire de recherche — fondements et limites

~5–7 pages

1.1 Structure et propriétés du BST

- Définition récursive : noeud, sous-arbre gauche (valeurs < noeud), sous-arbre droit (valeurs >= noeud)
- Opérations fondamentales : insertion, recherche, suppression — complexité $O(h)$
- Parcours in-order : produit les valeurs dans l'ordre croissant (intérêt pour les BDD)
- Visualisation pas à pas de la construction sur un petit jeu de données

1.2 Dégradation sur données pathologiques

- Cas de l'insertion triée : arbre dégénéré en liste chaînée, $h = n \rightarrow O(n)$ par opération
- Mesure empirique : hauteur après 1 000 éléments triés vs aléatoires
- Illustration : schéma comparatif arbre équilibré / arbre dégénéré
- Conséquence : un BST naïf est inutilisable en base de données

1.3 L'équilibrage AVL — rotations et facteur d'équilibre

- Facteur d'équilibre : $|\text{hauteur(gauche)} - \text{hauteur(droite)}| \leq 1$, maintenu à chaque noeud
- Les 4 rotations : droite (LL), gauche (RR), droite-gauche (LR), gauche-droite (RL) — schémas
- Pourquoi les rotations préservent la propriété BST (ordre in-order inchangé)
- Garantie $h \leq 1.44 \times \log_2(n)$ — complexité $O(\log n)$ assurée

1.4 Bilan : pourquoi l'AVL ne suffit pas en base de données

- Optimal en RAM, mais chaque accès noeud = un accès I/O coûteux sur disque
- Avec $h = \log_2(n)$, un index AVL sur 1 million de lignes = ~20 accès disque par recherche
- Transition : minimiser les accès disque → le B-tree

2.1 Le B-tree — généralisation pour le disque

- Principe : m enfants par noeud, chaque noeud = une page disque (4–16 Ko typiquement)
- Ordre m élevé (100–200) en BDD mais pas en RAM : coût d'accès disque vs comparaison CPU
- Propriétés : noeuds internes entre $\lceil m/2 \rceil$ et m enfants, arbre parfaitement équilibré
- Hauteur $O(\log_m n)$: avec $m=200$ et $n=1M$, $h \leq 3$ accès disque seulement

2.2 Le B+tree — la variante standard des SGBD

- Données uniquement dans les feuilles, noeuds internes = clés de routage seulement
- Feuilles chaînées : parcours séquentiel et requêtes BETWEEN sans remonter à la racine
- Noeuds internes plus petits → plus d'entrées par page → arbre moins haut
- Schéma comparatif B-tree vs B+tree : structure, emplacement des données, chaînage
- Pourquoi le B+tree est le standard (PostgreSQL, MySQL InnoDB, Oracle)

2.3 L'index Hash — concurrent ou complémentaire ?

- $O(1)$ pour les égalités pures (WHERE col = valeur)
- Inutilisable pour ORDER BY, BETWEEN, >, < (pas d'ordre sur les clés)
- PostgreSQL : index Hash disponible mais B-tree préféré par défaut

2.4 Impact des écritures — le coût caché de l'index

- Toute INSERT/UPDATE/DELETE doit aussi mettre à jour l'index
- Split de noeuds : opération coûteuse déclenchée périodiquement
- Index composites (colA, colB) : règle du préfixe — requête sur colB seul n'utilise pas l'index
- Sélectivité : index sur colonne à faible cardinalité peut être ignoré par l'optimiseur

3.1 Mise en place et génération du dataset

- Dataset synthétique : 1 million de lignes de logs de transactions (id, date, montant, client_id, statut)
- Génération Python (faker ou numpy) et chargement via psycopg2 ou COPY
- Distribution contrôlée : montants log-normale, statuts déséquilibrés (90% actif)

3.2 Lecture et interprétation des plans EXPLAIN ANALYZE

- Seq Scan : lecture séquentielle complète — quand et pourquoi PostgreSQL le choisit
- Index Scan : traversée B+tree puis accès heap pour chaque ligne
- Bitmap Index Scan : collecte des TIDs puis accès groupé — efficace pour plages larges
- Index Only Scan : toutes colonnes dans l'index, pas d'accès heap (le plus rapide)
- Métriques clés : actual time, rows, buffers, Planning vs Execution time

3.3 Benchmark — impact de l'index selon le type de requête

- Requête 1 (égalité) : WHERE client_id = X — avec et sans index B-tree
- Requête 2 (plage) : WHERE montant BETWEEN a AND b — chaînage des feuilles B+tree
- Requête 3 (tri) : ORDER BY date — index pour éviter le tri explicite
- Requête 4 (index ignoré) : faible sélectivité (statut = 'actif' sur 90%) → Seq Scan
- Graphiques : temps de requête / nombre de lignes pour chaque cas

3.4 Index composites et choix stratégiques

- Index composite (client_id, date) et test de la règle du préfixe
- Requête sur client_id seul → index utilisé ; requête sur date seul → index ignoré
- Compromis écritures/lectures : mesure du ralentissement des INSERT avec index
- Quand ne pas créer d'index : petites tables, faible cardinalité, forte volumétrie d'écriture

Conclusion

Synthèse et ouverture

~1–2 pages

Bilan

- Du BST naïf dégénérant au B+tree robuste : progression dictée par les contraintes hardware
- L'index n'est pas gratuit : accélère les lectures, ralentit les écritures, consomme de l'espace
- EXPLAIN ANALYZE comme outil indispensable pour valider les hypothèses d'optimisation

Ouverture

- Index partiels et fonctionnels dans PostgreSQL (WHERE statut = 'pending', LOWER(email))
- Index columnaires (BRIN) pour les données temporelles très volumineuses
- Bases de données analytiques : columnar storage (Parquet, DuckDB) et indexation

Questions-clés à préparer pour l'oral

- ? Pourquoi un index accélère les lectures mais ralentit les écritures ?
- ? Qu'est-ce qu'un Seq Scan, un Index Scan et un Bitmap Scan dans un plan PostgreSQL ?
- ? Pourquoi le B-tree a-t-il un ordre m élevé (100–200) en BDD mais pas en mémoire vive ?
- ? Dans quel cas un index sur (colA, colB) n'est-il pas utilisé pour une requête sur colB seul ?